

1. App Proposal: Postly

App Name

Postly

What the app is for, in one sentence

A daily postcard generator that allows a user to capture or upload a single image for the day, choose a template, add personal text, and archive it into a personal memory gallery and streak tracker.

Who is it for

- **Specific User:** Anyone who wants to maintain a daily photo-and-reflection habit without public social media pressure.
- **What they are trying to get done when they open it:** They want to quickly capture or upload today's photo, apply a nice template/caption to lock in their daily memory, check their current streak, and occasionally look back at past postcards.

Sections or routes this app needs

1. **/capture (Capture & Generator Screen):** Dedicated screen to upload today's single image, select a postcard template, add text/captions, and preview the final postcard before saving.
2. **/gallery (Postcard Gallery & Detail View):** Browsing screen displaying past postcards in a grid, allowing the user to click any card to view its high-resolution front and back, download it, or delete it.
3. **/streaks (Streak Calendar Dashboard):** A visual heatmap and calendar dashboard tracking daily capture history, creation streaks, and overall journaling statistics.
4. **/capsule ("Future You" Time Capsule):** A specialized vault view that locks memories for the future, surfaces random flashbacks, or allows reviewing messages left for a future date.

State: what data does the app hold?

- **Most Important Screen:** The Capture / Generator & Gallery screen.

Data	Shape (rough)	Who owns it (which component)	Changes when...
Postcards List	[{ id, imageUrl, caption, templateId, createdAt }]	App / Gallery Component	User saves a new daily postcard or deletes an existing one.
Today's Input State	{ imageFile, imageUrl, caption, selectedTemplate }	Capture Page Component	User uploads a file, selects a template, or types a caption.
Streak Stats	{ currentStreak, totalPostcards, historyDates: [] }	Dashboard Component	Calculated dynamically from the postcards dataset when loaded or updated.

What each screen contains

- **Screen:** Capture & Generator Screen (/capture)
 - **Block 1:** Header section with today's date and current creation status (whether today's postcard has already been made).
 - **Block 2:** Image upload / cloud dropzone widget (integrating Cloudinary).
 - **Block 3:** Template picker selector and text caption input form.
 - **Block 4:** Live interactive postcard preview container (front and back layout).
 - **Block 5:** "Save Postcard" submission action button.

Content you need to gather

- Sample/placeholder daily photos for initial UI testing and layout styling.
 - A Cloudinary cloud name and upload preset configuration for handling image hosting.
 - Color palette, typography choices, and 2-3 clean CSS layout templates for the postcard designs.
-

One risk

Coordinating the multi-step asynchronous workflow—specifically uploading the raw image file to Cloudinary on the front end, capturing the resulting secure URL, and successfully dispatching it along with the template configuration and caption in a single POST request to the Express backend without race conditions or state desynchronization.